

MemSnap: Automated Linux Memory Forensics with AI-Driven Threat Detection

Technical Report - Proof of Concept

Authors

Karim Elkhanoufi,
Karim Amaayou,
Moataz Bilah Aksaim

Supervisor

Prof. Manale Boughanja

CONTENTS

I.	Introduction	1
II.	System Architecture	2
III.	Infrastructure	3
	1. Lightweight Virtual Machine	3
	2. Memory Acquisition	3
	3. Volatility 3 and Symbol Table	3
IV.	Malware Components	4
	1. Rootkit (Diamorphine)	4
	2. Code Injection (LD_PRELOAD)	4
	3. Backdoor (Bind Shell)	4
V.	Feature Extraction	5
VI.	AI Model	6
	1. Dataset	6
	2. Classifier	6
VII.	Results and Limitations	7
	1. Backdoor Detection Issue	7
	2. Small Dataset	7
	3. Performance	7
VIII.	Demo / Showcase	8
	1. Infection & Capture	8
	2. Feature Extraction	8
	3. Model Training	8
	4. CLI Detection	9
	5. Web Interface	9
	6. Batch Testing	9
IX.	Conclusion	11
X.	Future Work	12
	Appendix A - Repository Structure	13
	Appendix B - Reproducing the Experiments	14

FIGURES

Figure 1 MemSnap pipeline overview	2
Figure 2 Repository Structure	13

TABLES

Table 1	Extracted forensic features and their relevance	5
Table 2	Training dataset - all combinations of threat labels	6

I. INTRODUCTION

Memory forensics is an essential technique for detecting advanced malware that operates entirely in RAM, bypassing disk-based defenses. However, manual analysis of memory dumps is time-consuming and requires deep expertise. **MemSnap** automates this process by:

1. Infecting a controlled Linux vm with malware components (rootkit, code injection, backdoor).
2. Capturing a compressed memory dump with [AVML](#).
3. Extracting forensic features using [Volatility 3](#).
4. Training a machine learning classifier to detect three threat categories.
5. Providing a **CLI** and a **web** interface for single-dump verdicts.

This report describes the design, implementation, and evaluation of the MemSnap proof-of-concept pipeline.

II. SYSTEM ARCHITECTURE

The pipeline consists of the following main stages:



Figure 1: MemSnap pipeline overview

All commands are dispatched through a unified entry point `memsnap`. Pre-built malware assets (kernel module and shared library) are stored in the `assets/` directory, so no compilation is required at runtime.

III. INFRASTRUCTURE

1. Lightweight Virtual Machine

A minimal Ubuntu 24.04 VM was prepared for the experiments. Unnecessary services, snap packages, and the graphical desktop were removed, leaving a headless system that idles at approximately **159 MB** of RAM. The hypervisor is configured with **512 MB** of memory, leaving ample room for malware execution and memory capture.

- **Kernel version:** 6.17.0-22-generic
- **Build tools:** git, build-essential, linux-headers-\$(uname -r) are installed for compiling the rootkit module.

2. Memory Acquisition

AVML (Azure VM Linux) from Microsoft is used to capture memory. It compresses dump files on-the-fly, reducing a 1 GB raw dump to approximately 180-200 MB. AVML is automatically downloaded into \$HOME/.local/bin/ on first use.

3. Volatility 3 and Symbol Table

The Volatility 3 framework (v2.28.0) is installed inside a Python virtual environment. A pre-generated ISF symbol table for kernel 6.17.0-22-generic is placed in assets/, enabling all plugins to function without additional setup.

IV. MALWARE COMPONENTS

Three distinct threat types are used to train the AI model. Each leaves a characteristic memory artifact that can be extracted by Volatility.

1. Rootkit (Diamorphine)

The open-source kernel module **Diamorphine** is pre-compiled for the VM's kernel. It hides itself from module listings, hides processes (e.g., PID 1) from `/proc`, and hooks syscalls such as `getdents64`. After loading, the module is invisible to `lsmod`, but Volatility's `linux.hidden_modules` plugin can detect it by comparing the kernel's module list with the list of loaded modules (or by scanning for hidden structures).

2. Code Injection (LD_PRELOAD)

A malicious shared library (`evil.so`) is injected into a `sleep 9999` process using the `LD_PRELOAD` environment variable. To make this injection detectable in memory dumps, a unique marker file `/tmp/injection_marker` is created and kept open by a dedicated `tail -f sentinel` process. Volatility's `linux.lsof.Lsof` plugin reads the kernel's `task_struct` → `files` → `fdtable` chain directly from physical memory, so it can see the open file descriptor even if the rootkit hides the process from `/proc`.

3. Backdoor (Bind Shell)

The `ncat` utility is installed on the VM, and a listening TCP socket is bound to port **4444** with a shell attached. This opens a backdoor that an attacker could use to gain remote access. The `linux.sockstat` plugin reveals all open sockets; a filter for `Source Port == 4444` and `State == LISTEN` identifies the backdoor.

V. FEATURE EXTRACTION

For each memory dump, the following Volatility 3 plugins are executed (in parallel where possible):

- `linux.pslist / linux.psscan`
- `linux.check_syscall`
- `linux.hidden_modules`
- `linux.sockstat`
- `linux.lsof.Lsof`

From their outputs, a set of numeric features is derived:

Feature	Description	Relevance
<code>hidden_module_count</code>	Number of hidden kernel modules detected	Perfectly separates rootkit dumps (value = 1) from clean ones (value = 0)
<code>has_injection</code>	1 if any process has an open file descriptor containing the string <code>injection_marker</code> , else 0	Perfectly separates injection dumps
<code>high_port_listeners</code>	Count of sockets listening on port 4444	Identifies backdoor dumps; however, one non-backdoor dump also shows this feature (see Section 7.1)
<code>total_sockets</code>	Total open sockets	Supplementary information
<code>hidden_proc_count</code>	Difference between PIDs seen by <code>psscan</code> and <code>pslist</code>	Potentially indicative of hidden processes (noisy in practice)
<code>hooked_syscall_count</code>	Number of hooked syscalls (from <code>check_syscall</code>)	Always zero on this kernel; retained for completeness

Table 1: Extracted forensic features and their relevance

The features are written to `features.csv`, which serves as the training dataset.

VI. AI MODEL

1. Dataset

A balanced dataset of 8 memory dumps was generated, covering every combination of the three threat categories:

Dump Name	Rootkit	Injection	Backdoor
memdump_..._clean.raw	0	0	0
memdump_..._rootkit.raw	1	0	0
memdump_..._injection.raw	0	1	0
memdump_..._backdoor.raw	0	0	1
memdump_..._rootkit_injection.raw	1	1	0
memdump_..._injection_backdoor.raw	0	1	1
memdump_..._rootkit_backdoor.raw	1	0	1
memdump_..._rootkit_injection_backdoor.raw	1	1	1

Table 2: Training dataset - all combinations of threat labels

The small size is sufficient for a proof-of-concept because the chosen features provide strong, nearly perfect separation for two of the three classes.

2. Classifier

A **multi-output Decision Tree** classifier (one tree per label) was chosen for its simplicity and interpretability. Using leave-one-out cross-validation (LOO-CV), the model achieves:

- **Rootkit detection:** 1.00 accuracy
- **Injection detection:** 1.00 accuracy
- **Backdoor detection:** ≈ 0.62 accuracy (exact match across all three labels)

The limited backdoor performance is due to a single confounding sample (see Section 7.1). The final model is saved as `assets/model.pkl` and loaded by both the CLI and web detection tools.

VII. RESULTS AND LIMITATIONS

1. Backdoor Detection Issue

The `rootkit_injection` dump exhibits `high_port_listeners = 2` (port 4444) even though no backdoor was active. This is an artifact of the experimental setup: the dump was captured shortly after a previous backdoor test, and a lingering socket remained in memory. As a result, the feature overlaps with true backdoor samples, causing the decision tree to misclassify that sample in 3 out of 8 LOO folds.

Solution (planned): add a more specific backdoor feature, such as checking for a process named `ncat` with the command-line arguments `-l -p 4444` using the `linux.pslist` plugin.

2. Small Dataset

With only 8 training examples, the model cannot be expected to generalise to unseen malware families or different kernel versions. Expanding the dataset with public sources (e.g., the CRYSYS Hidden LKM Rootkit dataset) and generating additional injection/backdoor dumps is a priority for future work.

3. Performance

Feature extraction takes about **4-5 minutes per dump** because each Volatility plugin runs sequentially. Parallelising the plugin execution and caching intermediate results would significantly reduce this time.

VIII. DEMO / SHOWCASE

1. Infection & Capture

Deploys the selected malware component and optionally triggers an AVML memory capture. The `-c` flag captures a dump immediately after infection.

```
memsnap infect --type rootkit_injection_backdoor -c
```

```
[*] Loading Diamorphine rootkit module...      [OK]
[*] Injecting evil.so via LD_PRELOAD...        [OK]
[*] Starting ncat bind shell on port 4444...    [OK]
[*] Capturing memory dump with AVML...         [OK]
    → dumps/memdump_20260506_110201_rootkit_injection_backdoor.raw.lz4
```

THE IMAGE `cmd_infect.png` GOES HERE.

2. Feature Extraction

Runs all Volatility 3 plugins against every dump in a directory and writes the resulting numeric features to `features.csv`.

```
memsnap extract dumps/ assets/features.csv
```

```
[*] Processing memdump_..._clean.raw           [1/8]
[*] Processing memdump_..._rootkit.raw         [2/8]
...
[*] Processing memdump_..._rootkit_injection_backdoor.raw [8/8]
[+] Features written to assets/features.csv
```

THE IMAGE `cmd_extract.png` GOES HERE.

3. Model Training

Trains the multi-output Decision Tree classifier with LOO-CV and saves the fitted model to disk.

```
memsnap train -i assets/features.csv -o assets/model.pkl
```

```
[*] Loaded 8 samples, 6 features, 3 labels
[*] Running leave-one-out cross-validation...
    Rootkit   accuracy : 1.00
    Injection accuracy : 1.00
    Backdoor  accuracy : 0.62 ← 1 confounding sample (see §7.1)
[+] Model saved to assets/model.pkl
```

THE IMAGE `cmd_train.png` GOES HERE.

4. CLI Detection

Given a single dump, extracts features on-the-fly and outputs a per-label verdict.

```
memsnap detect dumps/memdump_20260506_110614_injection.raw
```

```
Extracting features...
Rootkit    : not detected
Injection  : detected
Backdoor   : not detected
```

THE IMAGE cmd_detect.png GOES HERE.

5. Web Interface

The web application allows users to upload a memory dump and an optional custom model. Real-time progress is streamed via Server-Sent Events.

```
memsnap web
```

```
INFO:      Started server process [3821]
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

THE IMAGE web_upload.png GOES HERE.

THE IMAGE web_progress.png GOES HERE.

THE IMAGE web_results.png GOES HERE.

The interface also displays the extracted feature values, giving the analyst insight into the detection rationale.

6. Batch Testing

Runs detection across every dump in a directory and prints a summary verdict for each file.

```
memsnap test dumps/
```

```
=== memdump_..._clean.raw ===
Rootkit    : not detected
Injection  : not detected
Backdoor   : not detected

=== memdump_..._rootkit.raw ===
Rootkit    : detected
Injection  : not detected
Backdoor   : not detected

=== memdump_..._rootkit_injection_backdoor.raw ===
```

```
Rootkit    : detected  
Injection : detected  
Backdoor   : detected  
...
```

THE IMAGE `cmd_test.png` GOES HERE.

IX. CONCLUSION

MemSnap successfully demonstrates a fully automated pipeline for Linux memory forensic analysis with AI-driven threat detection. The key contributions are:

- **Reproducible malware deployment** using pre-built assets.
- **Robust injection detection** via a creative use of an open file descriptor marker and Volatility's LSOF plugin.
- **A clean, user-friendly CLI and web interface** suitable for both technical and non-technical users.

While the proof-of-concept is limited by a small dataset and imperfect backdoor detection, it provides a solid foundation for further research and development.

X. FUTURE WORK

- Add a backdoor-specific feature (process command-line analysis) to resolve the current misclassification.
- Expand the dataset with public memory samples and synthetic constrained-augmentation.
- Improve extraction performance through plugin parallelisation.
- Generate a formal PDF investigation report for each analysed dump.
- Explore more sophisticated models (Random Forest, XGBoost) once the dataset grows.
- Package the full tool (including the VM setup) into a single virtual appliance for educational use.

APPENDIX A - REPOSITORY STRUCTURE

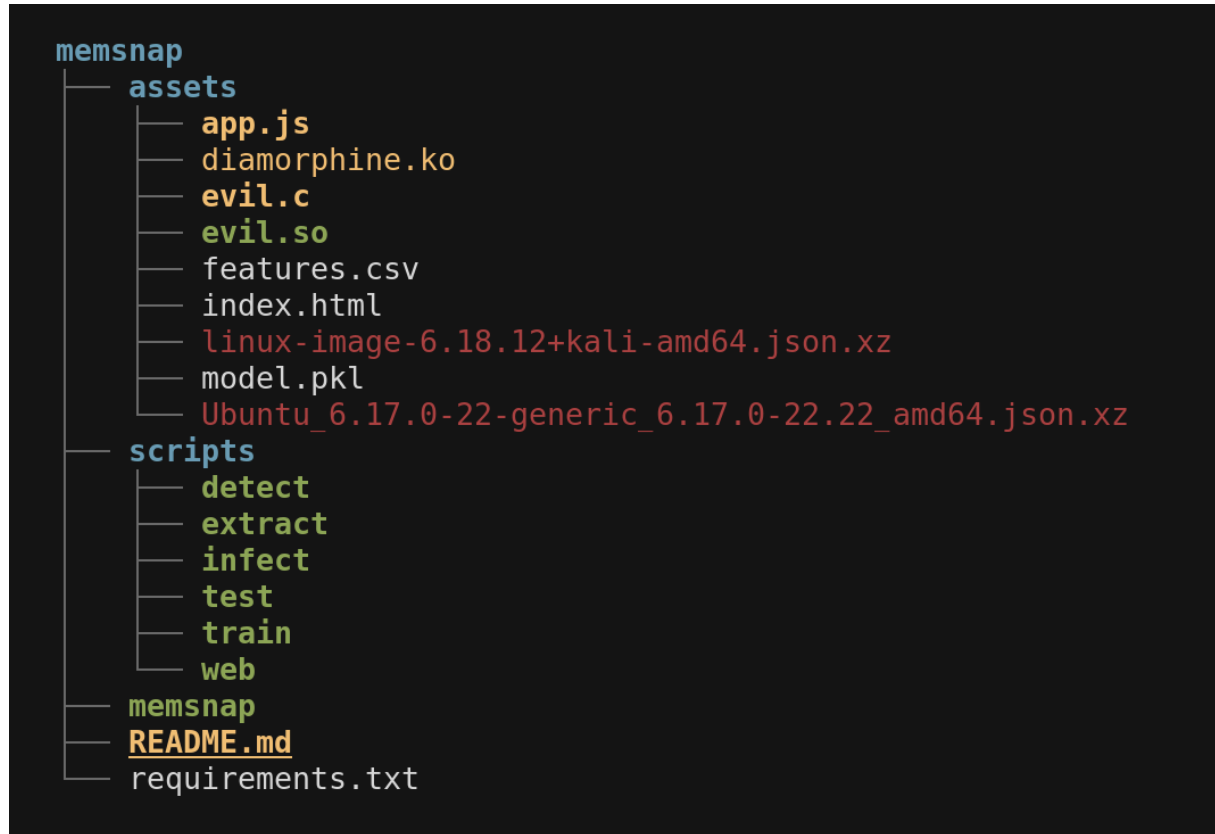


Figure 2: Repository Structure

APPENDIX B - REPRODUCING THE EXPERIMENTS

1. Set up a lightweight Ubuntu 24.04 VM (512 MB RAM).
2. Clone the repository and run the setup commands from the README.
3. Generate the dataset: `memsnap infect -c && ./generate_dumps.sh`
4. Extract features: `memsnap extract dumps/ assets/features.csv`
5. Train the model: `memsnap train -i assets/features.csv -o assets/model.pkl`
6. Test a dump: `memsnap detect dumps/...rootkit.raw`